Programming Lab 1: Introduction to Python

P Balamurugan,Urban Larsson,Sahil Wagh

July 14, 2024

1 Introduction

Welcome to the first programming lab! In this lab, we will cover the basics of Python programming, including for loops, while loops, and arrays (lists in Python).

2 Variable Declaration and Data Types

In Python, variables are dynamically typed, meaning you do not need to declare a variable's type before assigning a value to it. Here are some common data types:

- **Integer (`int`):** Whole numbers, e.g., `1`, `-5`, `100`.
- **Float (`float`):** Floating point numbers, e.g., `3.14`, `-0.5`, `2.0`.
- **String (`str`):** Textual data enclosed in quotes, e.g., `"Hello"`, `'World'`.
- **Boolean (`bool`):** Represents True or False values, e.g., `True`, `False`.

2.1 Example: Variable Declaration and Assignment

Let's declare and assign values to variables of different data types.

```
# Variable declaration and assignment
age = 25            # Integer
pi = 3.14           # Float
name = "Alice"      # String
is_student = True   # Boolean
```

Listing 1: Variable Example

2.2 Taking User Input

You can use the `input()` function to prompt the user for input. Here's an example:

```
# Taking user input
name = input("Enter your name: ")
age = int(input("Enter your age: "))
height = float(input("Enter your height (in meters): "))
is_student = input("Are you a student? (True/False): ").lower() ==
    'true'
```

Listing 2: Taking User Input

In the above example: - `input()` is used to take user input. - `int()` and `float()` are used to convert the input into integer and float types, respectively. - `.lower()` is used to convert the input to lowercase for case-insensitive comparison

2.3 Basic Operations

You can perform various operations on variables in Python:

- **Increment:** Increase the value of a variable by 1.

```
    i = 10
    i = i + 1
    print("Incremented i:", i)  # Output: 11

```

Listing 3: Increment Example

- **Decrement:** Decrease the value of a variable by 1.

```
    i = 10
    i = i - 1
    print("Decremented i:", i)  # Output: 9

```

Listing 4: Decrement Example

- **Division:** Divide the value of a variable by a number.

```
    i = 100
    i = i / 10
    print("Divided i:", i)  # Output: 10.0

```

Listing 5: Division Example

3 If-Else Statements

If-else statements in Python are used for decision making, allowing different actions based on conditions.

3.1 Example: Checking Even or Odd

Let's use an if-else statement to check if a number is even or odd.

```
# If-else statement example
num = 7
if num % 2 == 0:
    print(num, "is even.")
else:
    print(num, "is odd.")
```

Listing 6: If-Else Example

Dry Run:

- **Initialization:** `num = 7`
- **Condition:** Is `num % 2` equal to 0? No ($7 \% 2 = 1$).
- **Execution:** Print `7 is odd.`
- **Termination:** Exit the if-else statement.

4 For Loops

A for loop in Python is used for iterating over a sequence (such as a list, tuple, or string).

4.1 Example 1: Printing Numbers

Let's dry run a for loop that prints numbers from 1 to 5.

```
# For loop dry run example 1
for i in range(1, 6):
    print(i)
```

Listing 7: For Loop Example

Dry Run:

- **Initialization:** `i = 1`
- **Condition:** Is `i` less than or equal to 5? Yes.
- **Execution:** Print `i` (1).
- **Update:** Increment `i` to 2.
- **Condition:** Is `i` less than or equal to 5? Yes.
- **Execution:** Print `i` (2).
- **Update:** Increment `i` to 3.

- **... Repeat until `i` reaches 6.**
- **Termination:** Exit the loop.

Output:

```
1
2
3
4
5
```

4.2 Example 2: Iterating Over a List

Now, let's dry run a for loop that iterates over a list of names.

```
# For loop dry run example 2
names = ['Alice', 'Bob', 'Charlie']
for name in names:
    print("Hello, " + name + "!")
```

Listing 8: For Loop Example 2

Dry Run:

- **Initialization:** `name = 'Alice'`
- **Execution:** Print `"Hello, Alice!"`
- **Update:** Move to next element, `name = 'Bob'`
- **Execution:** Print `"Hello, Bob!"`
- **Update:** Move to next element, `name = 'Charlie'`
- **Execution:** Print `"Hello, Charlie!"`
- **Termination:** Exit the loop.

Output:

```
Hello, Alice!
Hello, Bob!
Hello, Charlie!
```

5 While Loops

A while loop in Python continues to execute a block of code as long as a given condition is true.

5.1 Example 1: Counting Down

Let's dry run a while loop that counts down from 5 to 1.

```
# While loop dry run example 1
i = 5
while i > 0:
    print(i)
    i -= 1
```

Listing 9: While Loop Example

Dry Run:

- **Initialization:** `i = 5`
- **Condition:** Is `i` greater than 0? Yes.
- **Execution:** Print `i` (5).
- **Update:** Decrement `i` to 4.
- **Condition:** Is `i` greater than 0? Yes.
- **Execution:** Print `i` (4).
- **Update:** Decrement `i` to 3.
- **... Repeat until `i` reaches 0.**
- **Termination:** Exit the loop.

5.2 Example 2: User Input Validation

Now, let's dry run a while loop that asks the user for input until a valid number is entered.

```
# While loop dry run example 2
number = 0
while number <= 0:
    number = int(input("Enter a positive number: "))
    if number <= 0:
        print("Invalid input. Please enter a positive number.")
```

Listing 10: While Loop Example 2

Dry Run:

- **Initialization:** `number = 0`
- **Condition:** Is `number` less than or equal to 0? Yes.
- **Execution:** Prompt user for input.
- **User Input:** Enter `-1`.

- **Validation:** Print `"Invalid input. Please enter a positive number."`
- **Condition:** Is `number` less than or equal to 0? Yes.
- **Execution:** Prompt user for input.
- **User Input:** Enter `5`.
- **Validation:** `number` is now 5, so exit the loop.
- **Termination:** Exit the loop.

6 Arrays (Lists)

In Python, arrays are called lists. They are used to store multiple items in a single variable.

6.1 Example 1: Modifying List Items

Let's dry run an example where we modify items in a list.

```
# List dry run example 1
my_list = [10, 20, 30, 40, 50]
for i in range(len(my_list)):
    my_list[i] += 5
print(my_list)
```

Listing 11: List Example

Dry Run:

- **Initialization:** `my_list = [10, 20, 30, 40, 50]`
- **Iteration:**
  - `i = 0`, `my_list[0] = 10 + 5 = 15`
  - `i = 1`, `my_list[1] = 20 + 5 = 25`
  - `i = 2`, `my_list[2] = 30 + 5 = 35`
  - `i = 3`, `my_list[3] = 40 + 5 = 45`
  - `i = 4`, `my_list[4] = 50 + 5 = 55`
- **Output:** Print `my_list [15, 25, 35, 45, 55]`
- **Termination:** Exit the loop.

6.2 Example 2: Filtering a List

Now, let's dry run an example where we filter out even numbers from a list.

```
# List dry run example 2
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_numbers = []
for num in numbers:
    if num % 2 == 0:
        even_numbers.append(num)
print(even_numbers)
```

Listing 12: List Example 2

Dry Run:

- **Initialization:** `numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`
- **Iteration:**
  - `num = 1`, skip (odd)
  - `num = 2`, append to `even_numbers`
  - `num = 3`, skip (odd)
  - `num = 4`, append to `even_numbers`
  - `num = 5`, skip (odd)
  - `num = 6`, append to `even_numbers`
  - `num = 7`, skip (odd)
  - `num = 8`, append to `even_numbers`
  - `num = 9`, skip (odd)
  - `num = 10`, append to `even_numbers`
- **Output:** Print `even_numbers [2, 4, 6, 8, 10]`
- **Termination:** Exit the loop.

6.3 Example: Sum of Numbers

Let's calculate the sum of numbers in an array.

```
# Sum of numbers in an array
numbers = [1, 2, 3, 4, 5]
sum = 0
for num in numbers:
    sum += num
print("Sum of numbers:", sum)
```

Listing 13: Sum of Numbers Example

Dry Run:

- **Initialization:** `numbers = [1, 2, 3, 4, 5]`, `sum = 0`

- **Iteration 1:** `num = 1`, `sum = 0 + 1 = 1`
- **Iteration 2:** `num = 2`, `sum = 1 + 2 = 3`
- **Iteration 3:** `num = 3`, `sum = 3 + 3 = 6`
- **Iteration 4:** `num = 4`, `sum = 6 + 4 = 10`
- **Iteration 5:** `num = 5`, `sum = 10 + 5 = 15`
- **Termination:** Exit the loop.

Output:

```
Sum of numbers: 15
```

7 Assignments

To practice the concepts covered in this lab, please complete the following assignments in Google Colab:

- Assignment 1: Basic Python Operations
  - Click here to open Assignment 1 in Google Colab